OlaFlow Case – SAMPLE

Lennon Ó Náraigh

September 18, 2025

Contents

1 Introducdtion

This document sets out how to run a simple simulation using OlaFlow in OpenFOAM. OlaFlow is custom package that can be used inside OpenFOAM to create simulations of motion in a wave tank [1]. In this example, OlaFlow is used to simulate a the waves in a 2D flume. OlaFlow does not come pre-installed with OpenFOAM, so the first step is to install OlaFlow, as described below. We describe the key steps in the installation of OlaFlow and the implementation of the simulation for users using `wsl` on the Windows Operating System; however, most of the commands for these tasks will be the same in Ubuntu.

2 Installation of OlaFlow

First, we install an Ubuntu virtual machine which will run in a Windows environment, this is done using the Windows Power Shell and the command:

```
wsl --install -d Ubuntu-18.04
```

We now initialize Ubuntu 18.04 by typing the following line into the Windows Power Shell terminal:

```
wsl -d Ubuntu-18.04
```

Next, we install the appropriate version of OpenFOAM, which is OpenFOAM6. The appropriate commands are obtained from https://openfoam.org/download/6-ubuntu/ and are repeated here:

```
sudo sh -c "wget -O - https://dl.openfoam.org/gpg.key |
   apt-key add -"
sudo add-apt-repository http://dl.openfoam.org/ubuntu
```

Update the apt package list:

```
sudo apt-get update
```

Install OpenFOAM6:

```
sudo apt-get -y install openfoam6
```

Update the installation:

```
sudo apt-get update
sudo apt-get upgrade
```

At the end of the installation, we must append the following line to our `bashrc` file (located in `Ubuntu-18.04\home\username`):

```
% . /opt/openfoam6/etc/bashrc
source /opt/openfoam6/etc/bashrc
```

We close the terminal and open a new one. We then start a new virtual Ubuntu session. By virtue of having modified the `bashrc`, OpenFoam 6 is automatically loaded. This can be checked by typing:

```
simpleFoam -help
```

We are now able to download olaFlow. This can be done using the `clone` command in GitHub. Alternatively, the installation can be downloaded as a zip file from https://github.com/phicau/olaFlow, this will produce the zip file `olaFlow-master.zip`, which we move in to the relevant directory and unzip. We next modify the permissions on the unzipped folder:

```
sudo chmod -R 777 olaFlow-master
```

We install 'make':

```
sudo apt-get install make
```

We can now use the OpenFOAM command 'all make' to generate the appropriate olaFlow executables. First:

```
cd olaFlow-master
./allMake
```

Then:

```
cd genAbs
./allMake
```

To run the various codes that generate transient boundary conditions in olaFlow, we require Python. Hence, we install `pip`:

```
sudo apt install python-pip
```

and finally, `numpy`:

```
sudo pip install numpy
```

Once these installations are complete, olaFlow is ready to be used.

3 OlaFlow tutorials

OlaFlow comes pre-installed with a number of tutorials, which can be found in the directory `olaFlow-master/tutorials`:

- CR35_dambreak
- baseWaveFlume
- breakwater
- currentWaveFlume
- irreg45degTank
- oppositeSolitariesFlume
- pistonFlumeABS
- setOlaFlume
- waveFloatingObject
- wavemakerFlume

- wavemakerTank

For instance, to run a flume model, we change into the relevant directory:

```
cd tutorials
cd wavemakerFlume
```

The relevant OpenFOAM case can be run using the appropriate file. For instance, for the piston wave maker, we type:

```
. runCasePiston
```

4 OpenFoam Directory Structure

We describe here the directory structure for the `SAMPLE` case in OlaFlow. This case is a customized version of the `wavemakerFlume` tutorial case which comes installed with OlaFlow (Section 3). The `SAMPLE` can be copied into the directory `olaFlow-master` and run from there, this takes care of all the dependencies. The directory structure for the `SAMPLE` is shown in Figure 1.

5 Running the SAMPLE simulation

The `SAMPLE` case is already set up for the computational domain shown in Figure 2. To understand this set-up in more detail, or to fine-tune the case, here are some key steps:

1. Fine-tune parameters, e.g.:
   - Physical parameters (surface tension, gas and liquid viscosities and densities), etc. in `constant/transportProperties`
   - Frequency at which snapshots of the simulation state are saved to files in `system/controlDict`
2. Set up a domain decomposition for execution of the code in parallel, if required, in `system/decompseParDict`
3. Customize the initial conditions in the the following files, if required:
   - `0.org/alpha.water`
   - `0.org/p_rgh`
   - `0.org/U`
4. Customize the mesh in the `system/directory`, if required. This can be done by making changes to the blockmesh, in `system/blockMeshDict` file.

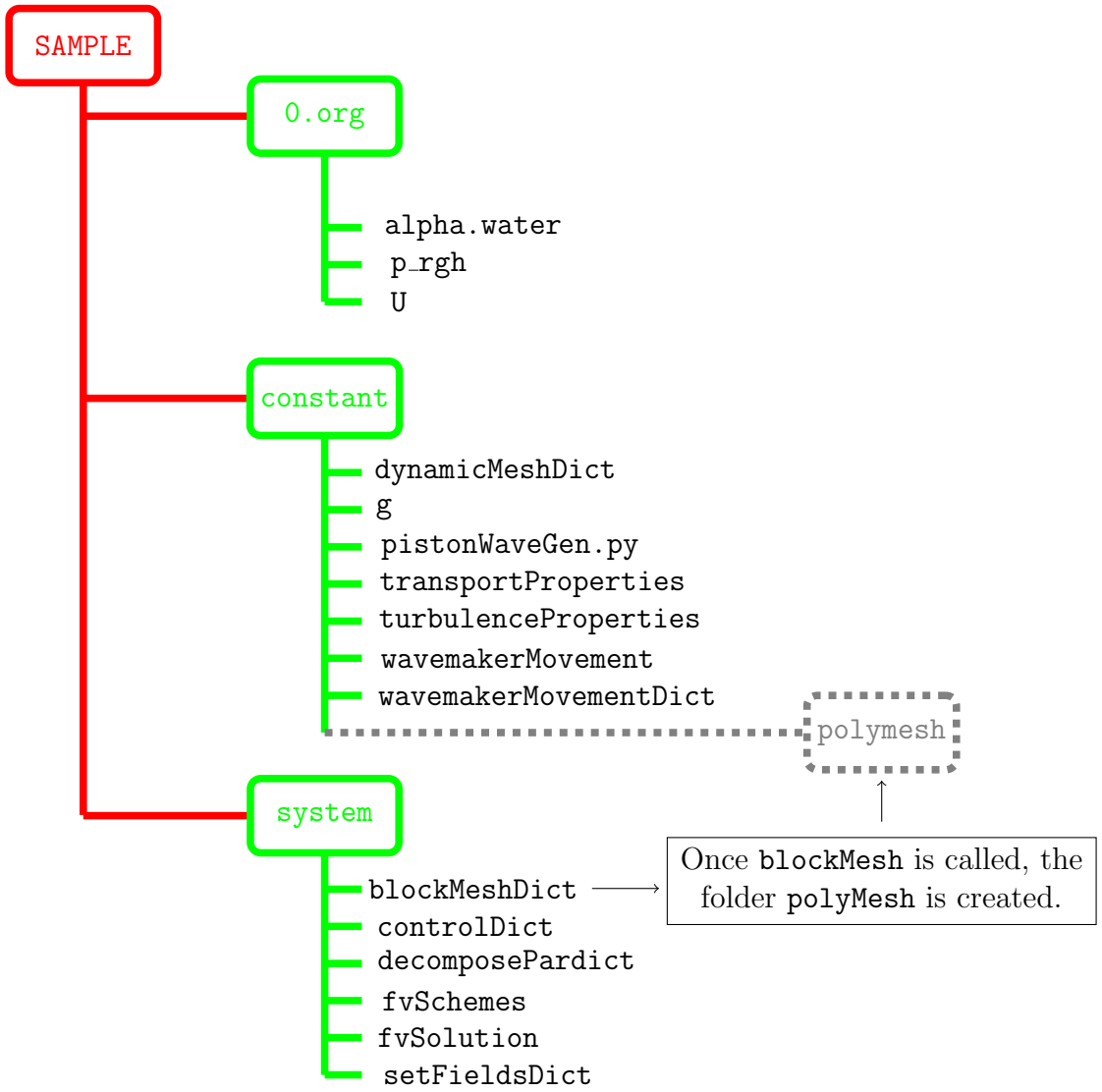

Figure 1: Directory structure for the OpenFoam simulation SAMPLE. The directory indicated by the grey dashed line is generated using the `blockMesh` utility.

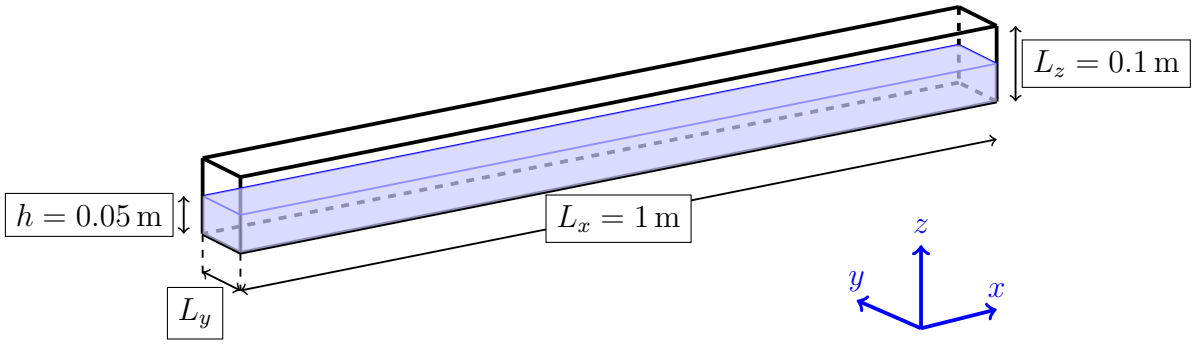

Figure 2: Schematic diagram showing the geometry of the `SAMPLE` case, together with the initial condition for the water level.

5.1 The dynamic boundary condition at $x = 0$

The dynamic boundary condition which describes the motion of the piston wavemaker is specified in the Python code `pistonWaveGen.py`. Here we highlight some of the key variables which the user may wish to modify:

- Period of piston: T
- Phase of oscillation: `phase0`
- Stroke length of piston: S

5.2 How to run the case

The bash script `runCasePiston` executes the code to simulate the piston wavemaker. This can be done by typing

```
. runCasePiston
```

at the command prompt. Here, we break down the different steps in this script for debugging purposes.

1. First, create the zero directory containing modified initial conditions:

   ```
   mkdir 0
   ```

2. Generate the blockMesh:

   ```
   blockMesh > blockMesh.log
   ```

3. Prepare the '0' folder:

   ```
   rm -r 0
   cp -r 0.org 0
   ```

4. Update the initial conditions:

   ```
   setFields > setFields.log
   ```

   Thus, for example, `0.org/alpha.water` has the volume-of-fluid function α equal to zero everywhere by default. Through the copy-paste operations above, the same is also true of the directory `0`. The dictionary `setFields` has instructions to overwrite this such that $\alpha = 1$ in the liquid and 0 in the solid. The command `setFields` executes these instructions and overwrites the previous initial conditions in the folder `0`.

5. The initial conditions at the dynamic boundary $x = 0$ are generated:

   ```
   cd constant
   python pistonWaveGen.py
   ```

6. The OlaFlow code is executed (command for serial mode):

   ```
   olaDyMFlow > olaDyMFlow.log
   ```

5.3 Output and Postprocessing

- Simulation results (velocity, pressure, droplet shape) are written to the **SAMPLE/** directory at different timesteps.
- Results can be visualized with ParaView (using the '.foam' file).

5.4 Key Notes

- Ensure OpenFOAM is installed and sourced before running.
- Simulations may take time depending on computing resources.
- Modify parameters in `system/controlDict`, `system/fvSchemes`, and `system/fvSolution` to experiment with settings.

References

[1] Pablo Higuera. phicau/olaFlow: CFD for waves. Zenodo. doi: 10.5281/zenodo.1297013, 2018.